

Customer Onboarding AI

Task 2 — Formalising the UI Model Contract

Beginner-friendly guide to creating a machine-checkable JSON Schema

1. What Task 2 Is About

Task 1 established the Customer Onboarding AI project foundation and documented the runtime AI model contract in prose. Task 2 takes the next focused step: convert that contract into a machine-checkable JSON Schema.

The important idea is simple: a document can explain the rules to people, but a JSON Schema allows software to check whether a model's structured output follows those rules.

Task 2 does not build the application. It creates a formal, machine-readable definition of the allowed UI specification.

2. Where Task 2 Starts

The source of truth for this task is the model contract created during Task 1:

`specs/model-contracts.md`

That document describes the expected UI specification, approved components, component properties, CTA actions, and restrictions on runtime model behaviour.

The output of Task 2 is:

`specs/schemas/ui-spec.schema.json`

3. Why Do We Need a JSON Schema?

Imagine that the runtime AI model produces this structured output:

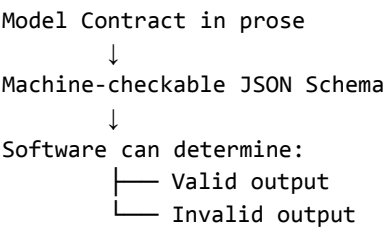
```
{
  "page": "customer_onboarding_landing",
  "components": [
    {
      "type": "hero",
      "title": "Welcome to Customer Onboarding"
    }
  ]
}
```

A human can look at this and decide that it appears reasonable. Software needs a precise definition of what is valid.

A JSON Schema provides rules such as:

- Which properties are required.
- Which values are allowed.
- Which component types are valid.
- How long strings may be.
- How many items an array may contain.
- Whether unexpected properties are permitted.
- What properties belong to each component.
- Which CTA actions are allowed.

4. The Core Concept



This is a key step in AI-native engineering because the runtime model is probabilistic. We do not simply assume that every model response follows the contract.

5. What JSON Schema Means for a Beginner

JSON Schema is a standard way of describing the shape and rules of JSON data. Think of it as a form template for software.

JSON Schema rule	Simple meaning	Example
required	This field must exist.	"page" must be present.
enum	Only these values are allowed.	"type" can only be an approved component.
const	The value must be exactly this.	"page" must identify the expected page.
minLength / maxLength	Controls text length.	A title cannot be excessively long.
minItems / maxItems	Controls array size.	A component list has a defined limit.
additionalProperties: false	Rejects unexpected fields.	A component cannot add arbitrary properties.
oneOf	The data must match one of several defined shapes.	Different component types have different property rules.
\$defs / \$ref	Defines reusable schema sections.	Component definitions can be referenced instead of duplicated.

6. The Exact Task 2 Prompt Given to Claude Code

The following prompt was used to ask Claude Code to formalise the existing model contract. The instruction deliberately prevents Claude Code from redesigning the contract or moving ahead into application development.

Proceed with the proposed next smallest task.

Formalise the UISpec model contract as a machine-checkable JSON Schema.

Create:

specs/schemas/ui-spec.schema.json

Use JSON Schema Draft 2020-12.

The schema must represent the contract currently defined in:

specs/model-contracts.md

Do not redesign the contract or introduce new capabilities.

Before writing the schema:

1. Read:

- CLAUDE.md
- docs/business-problem.md
- docs/architecture.md
- specs/spec.md
- specs/model-contracts.md

2. Identify any ambiguity or contradiction between the prose contract and what can be represented in JSON Schema.

3. If an ambiguity affects the schema design, document it clearly rather than silently inventing a rule.

The schema must cover:

- UISpec top-level structure
- Required fields
- Allowed component types
- Component-specific properties
- String length constraints
- Enum constraints
- Required properties
- Additional-property restrictions
- FeatureGrid child rules
- Hero child/container rules if defined by the contract
- InfoPanel child/container rules if defined by the contract
- CTA action enum/table
- Node-count limits where JSON Schema can express them
- Array size limits where applicable

Use JSON Schema capabilities such as:

- \$schema

- \$defs
- \$ref
- enum
- const
- required
- additionalProperties
- minLength
- maxLength
- minItems
- maxItems
- oneOf / anyOf where appropriate

Do NOT add application code yet.

Do NOT add runtime dependencies.

Do NOT build the React application.

Do NOT build the FastAPI application.

Do NOT integrate a model.

Do NOT create authentication.

Do NOT create document processing or risk functionality.

After creating the schema:

1. Validate that the schema itself is syntactically valid.
2. Check that every v0 component in the model contract is represented.
3. Check that every allowed CTA action is represented.
4. Check that the schema does not accidentally permit arbitrary component types.
5. Report any contract ambiguity discovered.
6. Summarize exactly what was created.

Stop after this task. Do not proceed to the validator or tests until explicitly instructed.

7. Understanding the Task 2 Prompt

“Formalise the UISpec model contract.” Claude Code is not being asked to invent a new contract. It is translating the existing written contract into a format that software can understand.

“Machine-checkable JSON Schema.” The result must be something a validation tool can use to determine whether a UISpec JSON document is valid.

“Use JSON Schema Draft 2020-12.” This selects a specific version of the JSON Schema standard so the schema has a well-defined vocabulary and behaviour.

“Do not redesign the contract.” This protects the boundary between specification and implementation. If the contract needs changing, that should be an explicit decision rather than an accidental change during implementation.

“Read the project documents first.” Claude Code must understand the business problem, architecture, system specification, and model contract before translating the contract.

“Identify ambiguity.” Prose can contain rules that are unclear or difficult to express formally. Claude Code should surface these issues rather than silently making assumptions.

“Component-specific properties.” A Hero should have Hero properties, a CTA should have CTA properties, and so on. The schema should not treat every component as an unrestricted object.

“Additional-property restrictions.” Unexpected fields should not automatically be accepted. Restricting properties helps prevent the model from inventing unsupported structures.

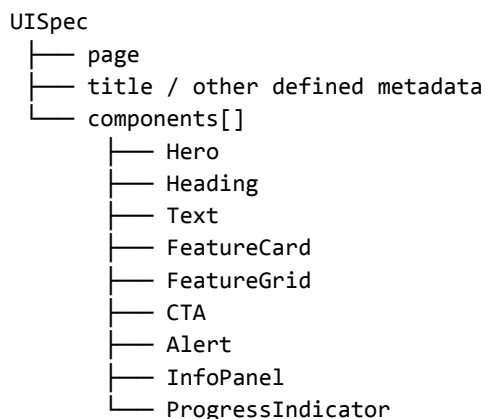
“Node-count and array-size limits.” The generated UI should have practical size limits. Some can be expressed directly in JSON Schema; others may require runtime validation.

“Do NOT add application code.” Task 2 is contract formalisation only. No React, FastAPI, model integration, database, or authentication should appear.

“Stop after this task.” The work is intentionally incremental. Claude Code should finish the schema task and wait for human review rather than automatically implementing later work.

8. What the Schema Needs to Represent

The schema should describe the UISpec structure already defined in the model contract. At a high level, the structure looks like this:



The exact properties and constraints must come from specs/model-contracts.md. The purpose of Task 2 is not to invent additional fields simply because they seem useful.

9. Approved Component Vocabulary

The v0 component vocabulary established for the landing page is:

- Hero
- Heading
- Text
- FeatureCard
- FeatureGrid
- CTA
- Alert

- InfoPanel
- ProgressIndicator

A major purpose of the schema is to ensure that an arbitrary component type cannot silently enter the system.

Valid:

```
{
  "type": "hero"
}
```

Not valid:

```
{
  "type": "execute_shell_command"
}
```

10. Why Component-Specific Rules Matter

A common beginner mistake is to create a schema that only says: “there is a components array.” That is not enough. Different components have different meanings and therefore different allowed properties.

CTA example

```
{
  "type": "cta",
  "action": "start_onboarding",
  "label": "Start Customer Onboarding"
}
```

The CTA action should come from the approved action table in the model contract. The model should not be able to invent arbitrary actions.

11. Why additionalProperties Matters

Suppose a model returns:

```
{
  "type": "cta",
  "action": "start_onboarding",
  "label": "Start",
  "run_this_code": "..."
}
```

If the schema accepts arbitrary additional properties, an unexpected field could pass structural validation. Using additionalProperties restrictions helps keep the data structure aligned with the contract.

This is not the same as saying JSON Schema provides complete security. It is one layer of control within a larger application boundary.

12. Why String and Array Limits Matter

AI-generated content can become unnecessarily large or repetitive. Constraints help keep the generated UI practical.

String limits: Prevent excessively long headings, labels or descriptions.

Array limits: Prevent a feature grid from containing an unreasonable number of items.

Node limits: Limit the total structural complexity of the generated UI where the rule can be expressed or separately enforced.

13. JSON Schema Cannot Express Every Rule

This is an important engineering point. JSON Schema is powerful, but it is not the entire runtime validation strategy.

Type of rule	JSON Schema	May require runtime logic
Field exists	Yes	Usually no
Allowed enum value	Yes	Usually no
Maximum array size	Yes	Usually no
Component structure	Yes, when representable	Sometimes
Complex cross-field business logic	Limited	Often yes
Security / authorization	No	Yes
Actual tool execution safety	No	Yes
Business decision correctness	No	Yes

Therefore, Task 2 should express the rules that naturally belong in the data schema and leave genuinely runtime-dependent rules documented for the appropriate validation layer.

14. Contract Ambiguity: Why We Ask Claude Code to Report It

Written specifications often contain statements that sound clear to a person but have multiple possible technical interpretations. For example, a phrase such as “Hero may contain children” needs a precise definition before software can enforce it.

The Task 2 prompt therefore tells Claude Code to identify ambiguity or contradiction instead of silently choosing a rule.

```
Prose requirement
  ↓
Is it precise enough?
├── Yes → encode it in schema
└── No  → report ambiguity
           rather than inventing behaviour
```

This is an important AI-native engineering practice: the coding agent should expose uncertainty instead of hiding it.

15. The Expected Task 2 Output

The expected result is a new machine-readable artifact:

```
specs/schemas/ui-spec.schema.json
```

The task should also produce a clear Claude Code summary stating:

- That the schema was created.
- That the schema itself was syntactically validated.
- Which v0 components were represented.
- Which CTA actions were represented.
- That arbitrary component types are not accidentally allowed.
- Any ambiguity discovered in the existing contract.
- That no application runtime code was added.

16. How to Think About the Result

Before Task 2

Model Contract



Human-readable rules



People can understand them

After Task 2

Model Contract



JSON Schema



Software can understand and check the structure

This does not make the AI deterministic. It makes the boundary around the AI more explicit and machine-checkable.

17. The Engineering Principle Behind Task 2

The runtime AI model is probabilistic. It may generate a correct structure, an incomplete structure, or something outside the intended contract.

Probabilistic model



Output



Machine-checkable contract



Can be classified as structurally valid/invalid

This is a fundamental AI-native engineering pattern: place deterministic controls around probabilistic components.

18. What Task 2 Does NOT Mean

- It does not mean the model itself becomes deterministic.
- It does not mean every business rule is solved by JSON Schema.
- It does not replace the model contract documentation.
- It does not build the UI renderer.
- It does not integrate an AI model.
- It does not create a React application.
- It does not create a FastAPI application.
- It does not create authentication.
- It does not implement document processing or risk assessment.
- It does not automatically mean the system is production-ready.

19. Task 2 Completion Checklist

- ☐ The existing model contract was read before schema creation.
- ☐ A JSON Schema Draft 2020-12 artifact was created.
- ☐ The UISpec top-level structure is represented.
- ☐ Required fields are represented.
- ☐ All approved v0 component types are represented.
- ☐ Component-specific properties are constrained.
- ☐ String constraints are represented where defined.
- ☐ Enum constraints are represented where defined.
- ☐ Additional properties are restricted appropriately.
- ☐ Container/child rules are represented where the contract defines them.
- ☐ CTA actions are constrained to the approved action table.
- ☐ Array/node limits are represented where JSON Schema can express them.
- ☐ The schema itself was checked for validity.
- ☐ Potential contract ambiguities were reported rather than silently invented.
- ☐ No runtime application code was added.
- ☐ No model integration was added.
- ☐ No authentication or unrelated functionality was added.

20. Key Beginner Lessons from Task 2

A written contract is useful, but a machine-checkable contract is stronger. People can read prose; software can directly validate a schema.

Do not let implementation silently change the specification. If the existing contract is ambiguous, identify the ambiguity rather than inventing a new rule.

Schema design is part of AI engineering. The schema defines a controlled boundary around structured model output.

Allow-listing is important. The schema should recognise the approved vocabulary instead of accepting arbitrary component types.

Not every rule belongs in JSON Schema. Some runtime, business and security rules need separate enforcement mechanisms.

Small tasks are intentional. Task 2 focuses on one artifact so the team can inspect and understand it before moving forward.

21. End of Task 2

Task 2 turns the previously documented UI model contract into a machine-checkable schema. The project now has a formal definition of the structure expected from the runtime UI-generation model.

Key idea: Convert AI expectations from human-readable rules into machine-checkable constraints wherever possible.

Task 2 — Clarifications & Learning Enhancements

The original Task 2 document is retained. This section adds a few clarifications to make the relationship between the model contract, JSON Schema, validation, and the eventual renderer easier for beginners to understand.

1. The Role of JSON Schema

JSON Schema is a machine-checkable representation of the UISpec contract. It does not make the AI deterministic. Instead, it creates a deterministic structural validation boundary around probabilistic model output.

2. Where Task 2 Fits in the Engineering Flow

Model Contract (human-readable rules)



JSON Schema (machine-checkable structure)



Validator (structural + runtime/cross-field checks)



Approved UISpec



Controlled React Renderer

3. Schema Validation vs Runtime Validation

JSON Schema	Runtime Validator
Required fields	Cross-field relationships
Allowed component types	Total node count
Enums	Serialized size limit
Data types	Context equality checks
String/array limits	Rules that JSON Schema cannot conveniently express
additionalProperties restrictions	Other runtime safety/business constraints

4. Why additionalProperties: false Matters

In simple terms: only properties explicitly defined by the contract are allowed. Anything else is rejected. This prevents the model from quietly introducing arbitrary fields or capabilities that the application has not approved.

5. Example of an Invalid Model Output

The model is not allowed to invent executable or operational components. For example:

```
{
  "type": "execute_shell_command",
  "command": "rm -rf ..."
}
```

This must be rejected because `execute_shell_command` is not part of the approved component vocabulary. The renderer never receives it as an approved UI component.

6. Contract Clarifications Identified During Formalisation

- Hero children: the prose was clarified to allow only CTA and/or Text within the defined limits.
- `meta.generated_for`: the schema uses the defined `ContextPayload` enums; exact equality with the input remains a validator responsibility.
- Empty strings: minimum length constraints were added so fields cannot be empty where strings are expected.
- `ProgressIndicator.current`: the schema provides a defensive range; the exact `current < len(steps)` relationship remains a validator responsibility.
- `campaign/locale` in `meta.generated_for`: because they are not part of the defined contract, `additionalProperties: false` rejects them.

These are documented contract clarifications discovered during formalisation. They are not new application capabilities.

7. What Task 2 Has Achieved

Before Task 2:

- The expected model output was primarily described in human-readable prose.

After Task 2:

- The expected `UISpec` structure can be checked automatically against a formal schema.

Result:

- AI output → deterministic structural validation boundary

8. Key Lesson for AI-Native Engineering

The schema does not make the AI deterministic. It makes the boundary around the AI output deterministic.

This distinction is important: the model remains probabilistic, but the software can refuse output that does not satisfy the contract. This is one of the foundations of building reliable model-driven applications.

9. Scope Reminder

This document remains intentionally limited to Task 2. It does not implement the validator, React renderer, FastAPI service, model integration, authentication, document processing, risk functionality, or production infrastructure.